

操作手冊

目的：

針對 **Satnogs** 上面的訊號，進行處理與分析。分別有波型以及人工解碼的流程，讓後續人員能夠自行操作，並檢驗。

目錄

Chapter 1	專案架構介紹.....	3
1.1	專案介紹.....	3
1.2	使用專案前的準備.....	4
Chapter 2	封包檢驗.....	5
2.1	下載.ogg 檔案.....	5
2.2	確認封包狀態.....	5
2.3	一些錯誤的範例.....	7
Chapter 3	Excel 解讀.....	9
3.1	確認封包閾值.....	9
3.2	取得互動 Excel	10
3.3	進階分析技巧.....	11
Chapter 4	捷克訊號.....	14
4.1	轉檔.cs16 -> .wav	14
4.2	分析及解碼.....	14
4.3	Excel 人工解析	15
Chapter 5	總結.....	17

Chapter 1 專案架構介紹

此章節著重於專案架構的介紹，釐清如何使用，以及知道如果有任何想要優化的時候可以從何處著手，但詳細內容在此文件中不會詳細提及，任何詳細內容都在各個專案資料夾中的 **readme** 有所描述。

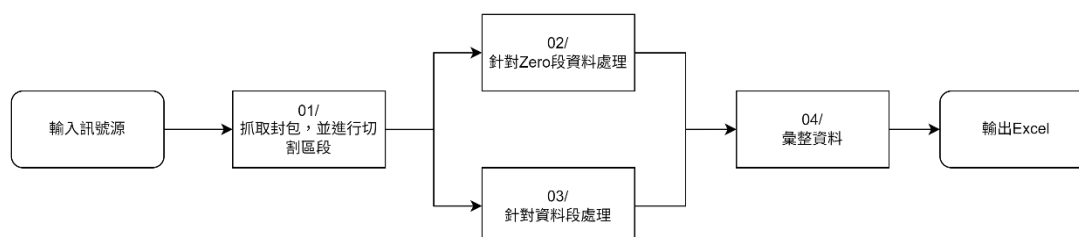
1.1 專案介紹

專案連結：<https://github.com/JacyyChang/scionx-decode-writeup>

這份專案主要用途在於解開 SCIONX 訊號封包，並盡可能地解開訊號內容，專案中有 4 份資料夾，就純粹使用中，主要會用到的功能為 01 以及 04 的資料夾，功能與檔名撰寫如下：

資料夾	主題	使用
01_frame_detection/	自動偵測封包起點（正規化互相關），再把整段訊號切成 header/data/zero-run/FCS 段落	Yes
02_zero_run_baseline/	自動偵測每個 frame 之後，用 decision-directed baseline restoration + 兩種判決方法，標出 zero-run 段落裡孤立的「1」	N/A
03_data_segment_processing/	自動偵測每個 frame 之後，從已知的 header bit 校正出固定偏移閾值：顯示不對稱性本身（03a）+ 套用判決線（03b）	N/A
04_Beacon/	不是診斷方法，而是 beacon telemetry 欄位格式參考資料	Yes

如果要理解處理步驟，可以參考下面的圖式，但切記這只是方便理解，真實資料並非真的連續的經過這些方式，純粹代表著訊號處理解法以及我解開的步驟。



1.2 使用專案前的準備

再使用這份 Repo 的時候，請先執行一下代碼：

```
1 pip install -r requirements.txt # numpy, matplotlib, soundfile,
  openpyxl
```

確保相關套件都有安裝到

接下來如果要執行，就可以切到對應的專案目錄下，並執行相對應的 python 檔案

```
1 cd 01_frame_detection # 或 02_.../ 03_.../ 04_Beacon/
2 python 01_frame_detection.py # 換成你想跑的腳本
```

Chapter 2 封包檢驗

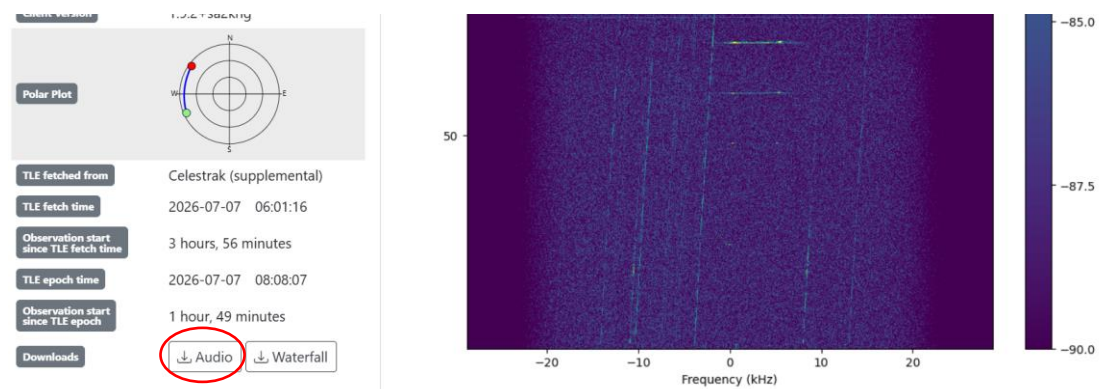
這個階段主要目的是分析 Satnogs 上面的 .ogg 檔案來分析是否是我們 SCIONX 的封包，並確認這個封包狀態是否良好。

2.1 下載.ogg 檔案

首先先去 Satnogs 任一個接收紀錄頁面，找尋 Audio 音檔

本次我使用是：<https://network.satnogs.org/observations/14459039/>

可以到介面下方點及 Audio 下載，如圖：

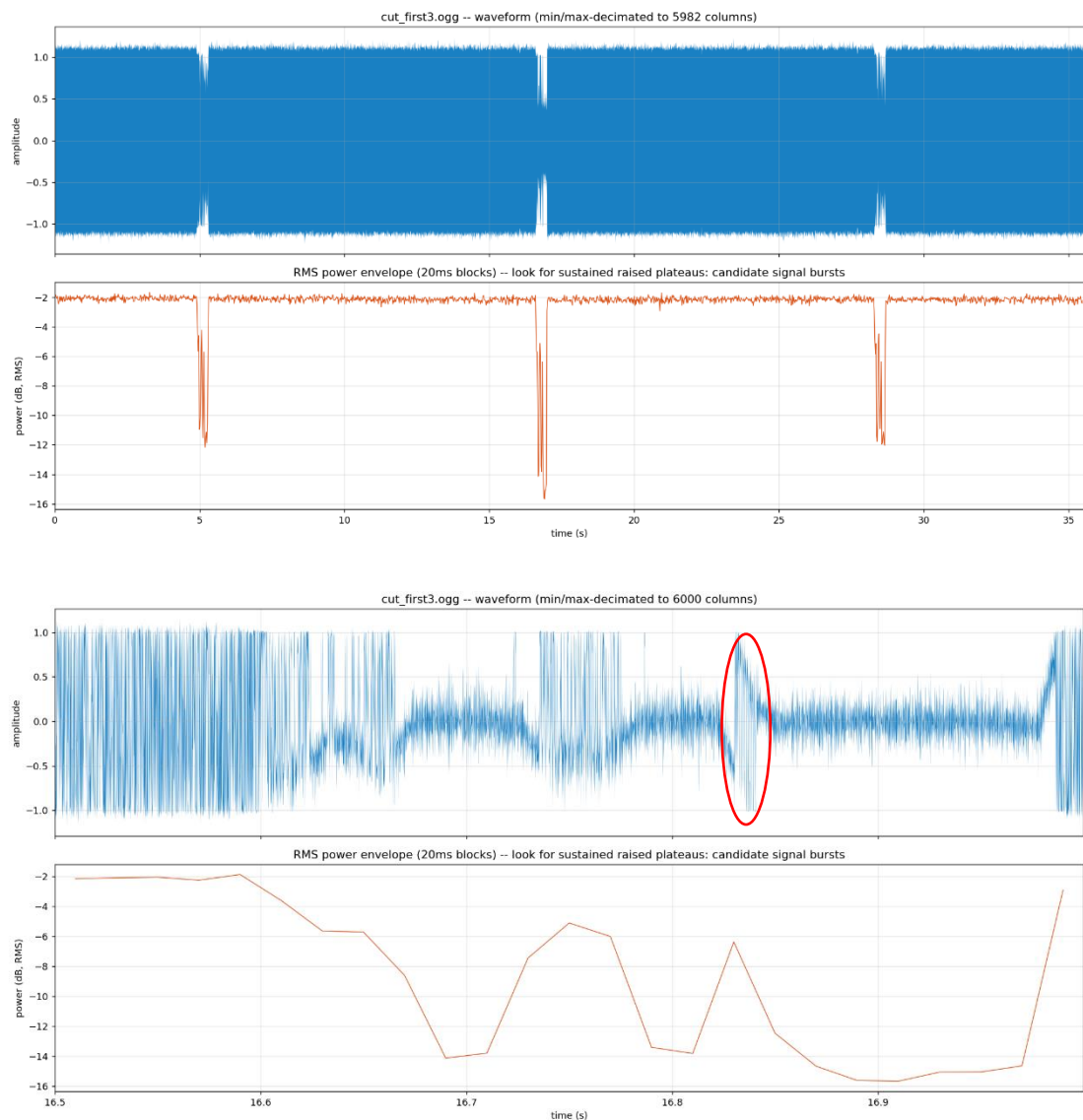


2.2 確認封包狀態

首先可以執行以下的指令，可以先觀察所有狀態，再去針對要看的區間做調整：

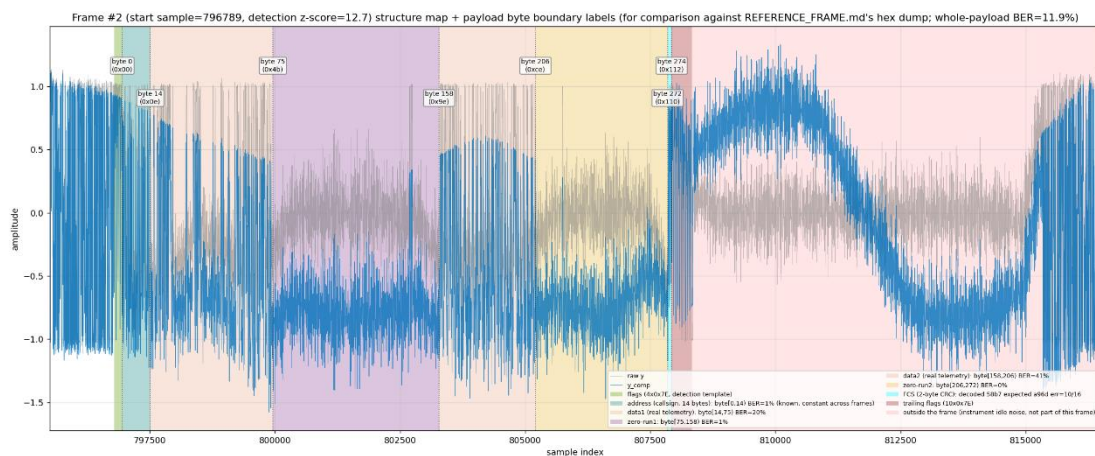
```
1 python 01a_waveform_power_overview.py path/to/long.ogg #  
   整份，會先印出總長度  
2 python 01a_waveform_power_overview.py path/to/long.ogg --start 30 --end  
   90 # 只看 30~90 秒
```

當執行第一個的時候，結果應該類似如下，如果 power 有下降(單純讓我辨識有封包的狀態出現，不代表任何真實的物理意義)，可以在放大區間做觀察：



這張圖是正常的圖片，可以看到有三段趨近於 0 的區間，這是正常現象，因為這段本身封包就是沒有送東西，最後面的則是沒有再發送任何訊號，然後還有特色的 7E 結尾串(紅色圈圈)。理論上封包都會長類似的情況。這個就有機會在最後透過人工辨識的方式去的部分衛星資訊。這樣去跑後面的 04 資料夾才有意義。

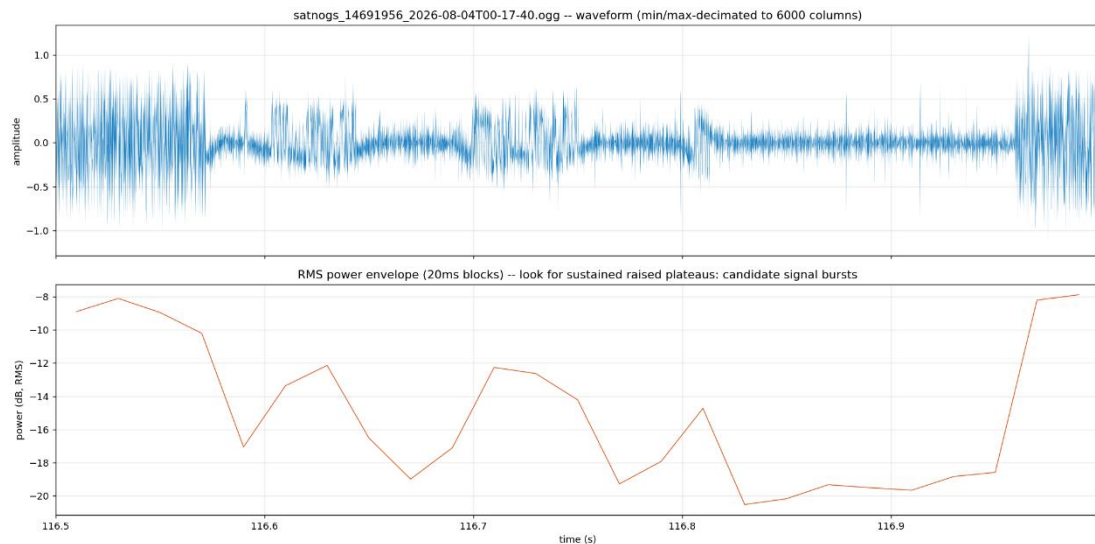
以下是透過另一個程式(01 資料夾)跑出來的結果，可以更詳細的知道封包波型架構：



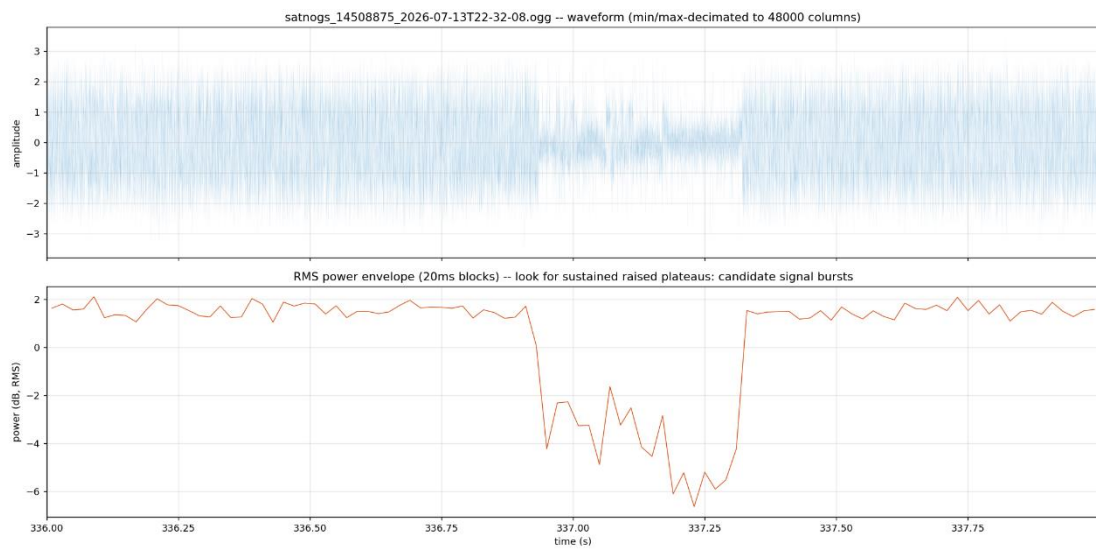
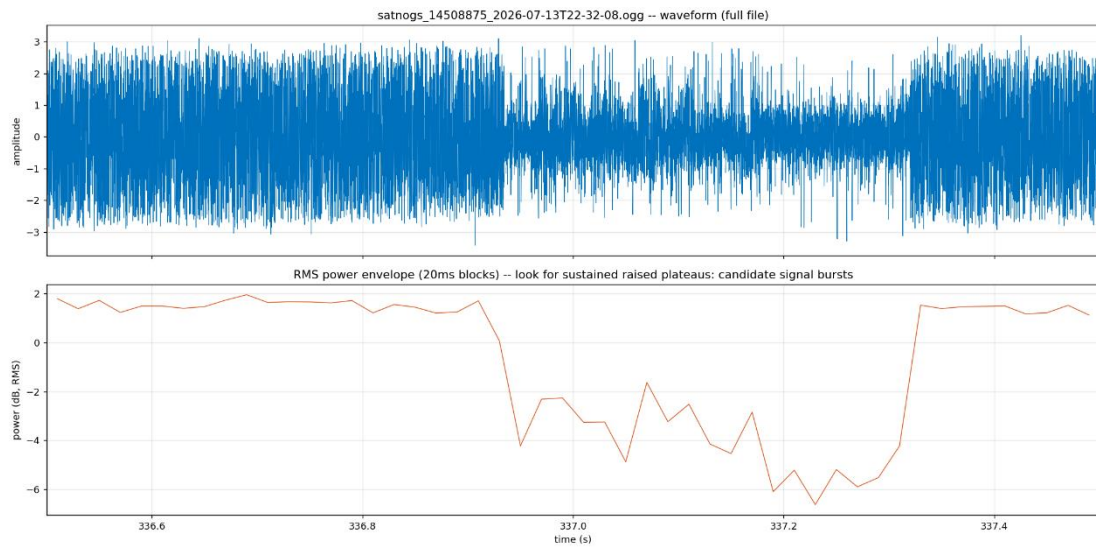
2.3 一些錯誤的範例

絕大多數的封包是可以知道是 SCIONX 的訊號，但是功率普遍過於弱小，以及似乎有阻抗問題(待調查)。

這個就是有阻抗問題的可能，因為前半部的資訊消失了。



或是根本看不出來，下面兩張圖是我們的，下面一頁的上圖是原始的，下面一頁的下圖則是 Downsample，如果 Downsample 的話可以看到趨勢，但這基本上是解不開的，所以即便看到是 SCIONX 訊號，但也會出現無法解的問題



Chapter 3 Excel 解讀

這篇關於如何使用 04 的資料夾，也是可以來判讀資料的部分，但由於通訊上的缺陷，所以需要人工來判讀。現在訊號處理兩個最大的問題：

1. 無法確切知道 bit 到底是 0 還是 1
2. HDLC Frame 特有機制是每 5 個 11111 後面補一個 0，會導致後續的 bit 全部 shift

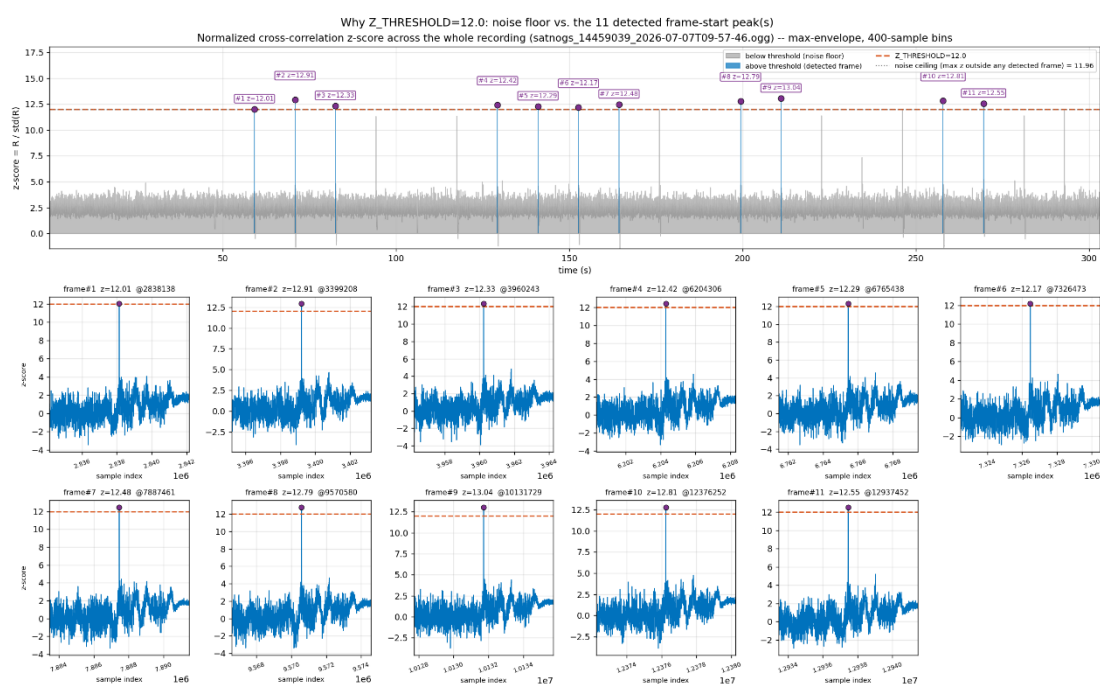
這兩個問題會互相糾結再一起，像是假設連續的 5 個 1 以及後續的 0 有個是不確定到底是 0 還是 1，就會出現這個後續的 0 是否是填充的 bit，需要拿掉，而這些問題就是需要人工判斷。

3.1 確認封包閾值

首先，如果確認了封包是有機會被擷取到的(參考 Chapter 2)，就可以先執行一下的指令：

- 1 # 1. 先畫出這份錄音的 z-score 曲線，打開 Figure/04a_zscore_<檔名>.png 看
- 2 python 04a_zscore_visualization.py path/to/other.ogg

會得到類似以下的結果：



到此你會看到一條水平線紅線(我稱作閾值)，這是用來確認那些封包可以被記錄到 excel，峰值處就是透過封包前面的呼號做 coorelation 取得的，所以有 peak 就代表有機會擷取出 excel。閾值可以自行調整，我自己偏好使用 10，只要有大於 10 的就會記錄成 excel。調整方式如下：

```
1 python 04a_zscore_visualization.py path/to/audio.ogg --z-threshold 12.0
```

3.2 取得互動 Excel

解下來就可以針對要分析的.ogg 檔案來取得 excel 檔案，執行一下指令：

```
1 # 3. 用同一組 frame/threshold 產生互動工作簿
2 python 04d_destuff_interactive_ver2.py path/to/audio.ogg --z-threshold 12.0
```

這個指令會把在門檻之上的都擷取成 Excel。

而取得到的 Excel，則會有許多頁籤，只需要關注 Rawbits and Beacon Decode，如果調整 Rawbits，後面的 Beacon Decode 就會自己跟著調整。

而 Excel 的區間顏色代表著，會採取這個模式請詳讀 Readme 的處理方式：

- 藍色 = header (Dest/Src address、Control、PID)
- 桃色 = data1
- 粉色 = data2
- 灰色 = zero-run1
- 深灰色 = zero-run2
- 紫色 = FCS
- 綠色 = 7E

首先如果要調整 0 or 1，可以看到 LowConfidence 這個選項，如果寫 Yes 就代表程式無法確切判定是 0 還是 1，所以可以調整 FlipThisBit 的選項。相對的，如果出現可能是 stuffbit，CandidateStuffPoint 這個欄位就會寫 Yes，就可以去調整 ExcludeThisBit 這個選項，後面整個都會被 shift。

BitIndex	RawBitValue	LowConfidence	CandidateStuffPoint	ExcludeThisBit	FlipThisBit	EffectiveBitValue
480	0	No		No	No	0
481	0	No		No	No	0
482	1	No		No	No	1
483	0	No		No	No	0
484	0	Yes		No	No	0
485	1	No		No	No	1
486	1	No		No	No	1
487	0	No		No	No	0
488	0	No		No	No	0
489	1	No		No	No	1
490	1	No		No	No	1
491	1	No		No	No	1
492	1	No		No	No	1
493	1	No		No	No	1
494	0	No	Yes	No	No	0
495	1	No		No	No	1
496	1	No		No	No	1
497	0	No		No	No	0
498	1	No		No	No	1
499	0	No		No	No	0
500	1	No		No	No	1

使用說明
RawBits
_DestuffedBits
Beacon Decode
CRC_Check
(+)

3.3 進階分析技巧

當然這裡面有幾個分析的技巧：

1. Zero 段的資料如果出現 1 的可能，可以檢查附近是否有其他 1 的存在，如果附近都沒有，通常是錯誤的。
2. 有多少個填充 bit 可以從 7E 位置得知，這也要人工判決。

關於第 2 點需要先看到 Excel 表的最後面，因為尾段有 10 個 7E(01111110)不斷重複，所以可以看到相關的狀態，像是下面的圖就可以很清楚地發現 2198 這個 BitIndex 是 7E 的開頭，而到尾段開始 2192，中間有 6 個 bit 的存在，就說明這個封包有 6 個填充 bit 的存在。

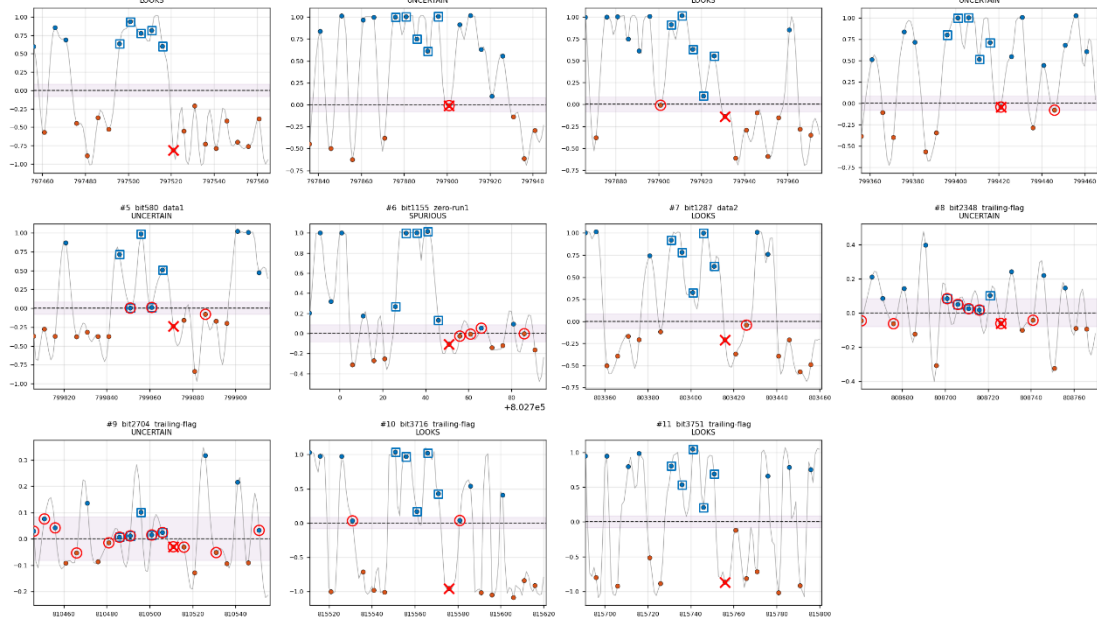
BitIndex	RawBitValue	LowConfidence	CandidateStuffPos	ExcludeThisBit	FlipThisBit	EffectiveBitValue	DestuffedSeqNum	SegmentGuessed
2187	1	No		No	No	1	2188	FCS
2188	1	No		No	No	1	2189	FCS
2189	0	No		No	No	0	2190	FCS
2190	1	No		No	No	1	2191	FCS
2191	0	No		No	No	0	2192	FCS
2192	1	No		No	No	1	2193	trailing
2193	0	No		No	No	0	2194	trailing
2194	1	No		No	No	1	2195	trailing
2195	1	No		No	No	1	2196	trailing
2196	0	No		No	No	0	2197	trailing
2197	1	No		No	No	1	2198	trailing
2198	0	No		No	No	0	2199	trailing
2199	1	No		No	No	1	2200	trailing
2200	1	No		No	No	1	2201	trailing
2201	1	No		No	No	1	2202	trailing
2202	1	No		No	No	1	2203	trailing
2203	1	No		No	No	1	2204	trailing
2204	1	No	Yes	No	No	1	2205	trailing
2205	0	No	Yes	No	No	0	2206	trailing
2206	0	No		No	No	0	2207	trailing
2207	1	No		No	No	1	2208	trailing
2208	1	No		No	No	1	2209	trailing
2209	1	No		No	No	1	2210	trailing
2210	1	No		No	No	1	2211	trailing
2211	1	No		No	No	1	2212	trailing

另外一個確認方式，是透過一下指令去觀察填充 bit 附近的波型，因為通常一包裡面有許多可能是填充 bit 的可能，通常會超過尾段自己判決出的數值。

- # 2. 看 04b 自己那套獨立、不做 destuff 的分析，找到你關心的那個 frame
- # 有幾個 bit-stuffing 移除事件，以及每個事件的上下文
- # （波形視窗、段落、是否低信心）
- `python 04b_stuffing_events.py path/to/audio.ogg --frame 2 --z-threshold 12.0`

這可以看底幾個 frame，bit stuff 填充可能的波型，可以見下圖：

Frame#2 (start sample=796789): every stuffed-0 removal event during destuffing -- squares=the 5 ones, red X=dropped 0, red ring= $|y| \leq \text{margin}$

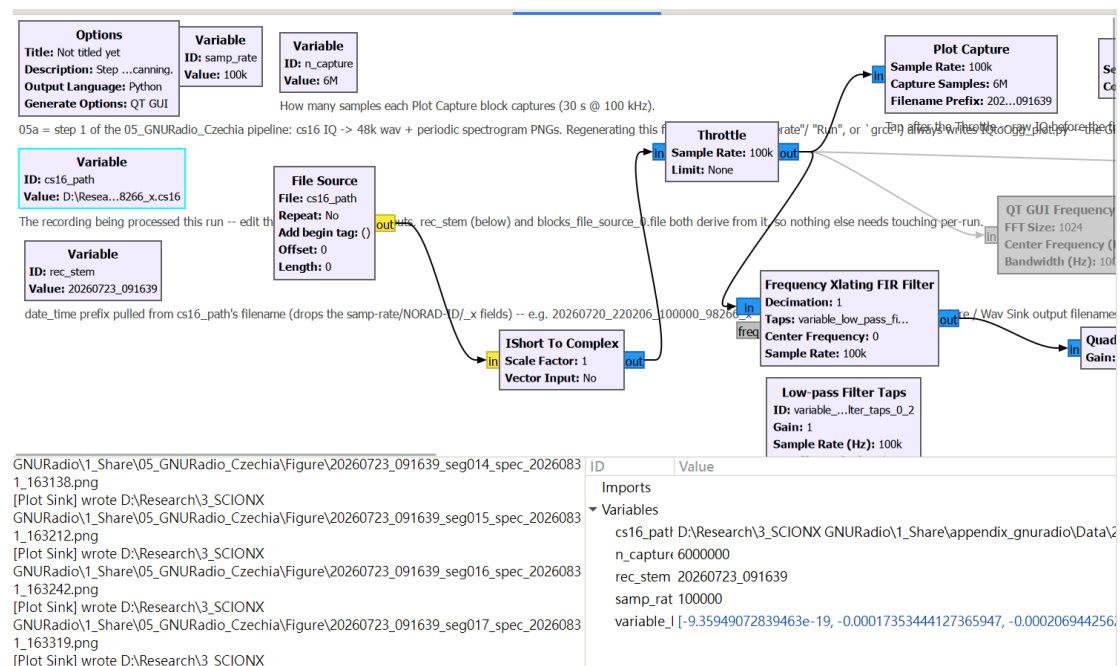


Chapter 4 捷克訊號

捷克的訊號錄製檔案使用.cs16 檔案進行錄製，所以分析的時候先轉到.wav 檔案，並做後續的分析，但這個分析的難度比起從 Satnogs 下載的還要好，Zero-run 的地方並沒有像 Satnogs 的 DC block 濾掉，保持很好的原始訊號。以下已 0723 的訊號檔案為主要範例。

4.1 轉檔.cs16 -> .wav

首先要打開 GNURadio 檔案(05a_IQtoOgg_plot.grc)，然後更動 Variable cs16_path 的路徑，指向捷克訊號的檔案位置。執行後會生成頻譜瀑布圖片，每 30 秒計一張圖，存在同個資料夾下的/Output 資料夾中，並說明第幾個 Seg，可以用來檢查訊號，然後也會儲存.wav 檔案。

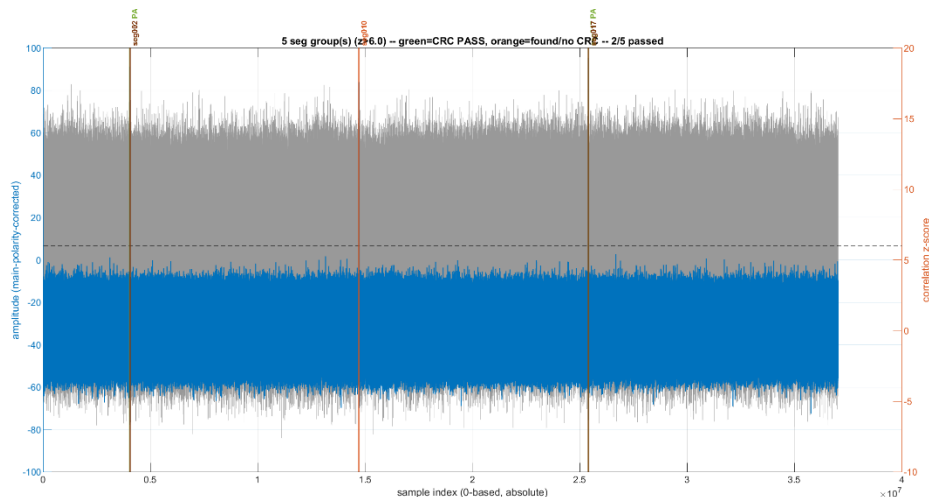


4.2 分析及解碼

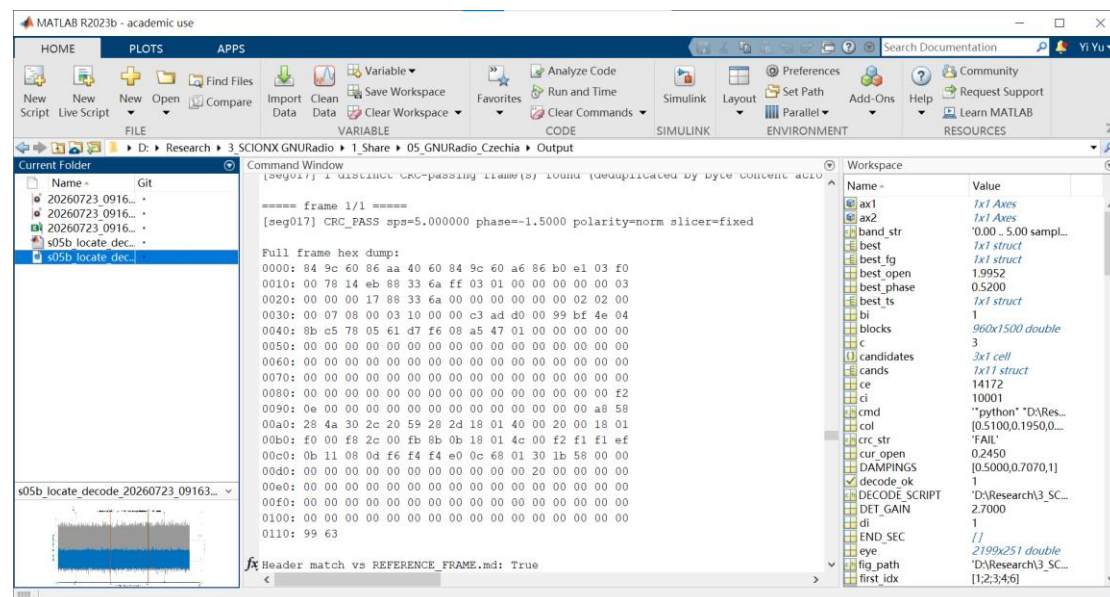
使用 Matlab 程式(s05b_header_locate_decode.m)，記得在更改檔案位置

```
1 %% ---- configuration -- edit these for a new recording ----
2 WAV='D:\Research\3_SCIONX
  GNURadio\1_Share\05_GNURadio_Czechia\Output\20260723_091639_48k.wav';
```

之後會生成出圖：



可以用這個迅速找到哪裡偵測到 Frame，同時也會標上 Seg 多少，可以查證，上面會寫 PA 代表解碼有過，同時 MATLAB Command Window 會顯示結果跟成功的封包數據，如下圖：



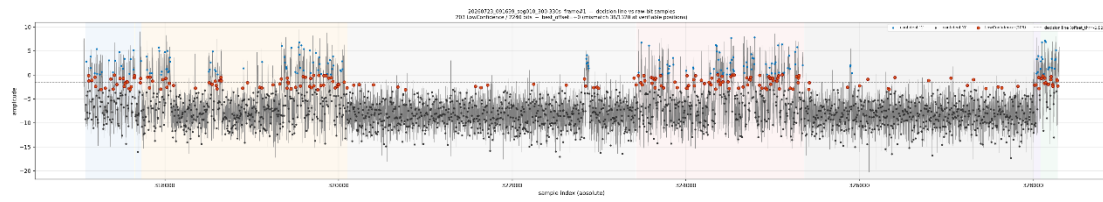
4.3 Excel 人工解析

但是像剛剛中間的 Seg10 封包解不開，則會需要透過之前 04 的土炮方式解開訊號，執行方式如下：

```
1 python 05c_destuff_interactive.py Output/<rec_stem>_48k.wav --seg 10
```

Rec_stem 是指每個.wav 前面的日期代號，簡單來說就是.wav 檔案就了，然後指定要分析哪個 Seg 即可，現在是指定 10，程式碼就會去搜尋那個區域，並抓出

來處理成 Excel，依樣會放置在/Output 資料夾中。同時也會產出決策的圖：



方便檢查決策位置。

Chapter 5 總結

由於訊號的卻失，許多部份需要人工判斷，目前此訊號處理是目前可能最佳方法。藉由這份文件的說明，後續的人員可以自行操作，檢驗 **Satnogs** 上的訊號，並取得封包。

捷克的訊號檔案有一半機率相當好，可以解開訊號，但是目前一過 8 月，就會遇到阻抗問題。